

02 AOU LowLevel Design

- [AOU CORE — Low-Level Design \(LT\)](#)
 - [1. Module API](#)
 - [2. Internal architecture](#)
 - [3. TLM contract](#)
 - [4. CCI parameters](#)
 - [5. SMC / SMU VP wiring](#)

AOU_CORE — Low-Level Design (LT)

1. Module API

```
namespace aou {
class aou_core : public sc_module {
    // CCI: rp_count (immutable), access_delay_ns, bridge_delay_ns
    tlm_utils::simple_target_socket<aou_core>      apb_socket; // 32-bit
    sc_vector<simple_target_socket_tagged<aou_core, 64>> axi_s; // TX in
    sc_vector<simple_initiator_socket<aou_core, 64>> axi_m; // RX out
    sc_in<bool> fdi_active_i;
    sc_out<bool> irq_o;

    void connect_peer(aou_core* peer);
    bool is_enabled() const;
};
}
```

2. Internal architecture

Block	Mechanism
APB decode	Switch on offset; plain <code>regmodel::Register32</code> storage
Activation	Flags + <code>try_activate</code> / <code>try_deactivate</code> / <code>peer_activate_ack</code>
AXI path	<code>b_transport</code> on <code>axi_s[id]</code> → peer <code>axi_m[dest_rp]</code>
IRQ	<code>update_irq()</code> drives <code>irq_o</code> from activate/deactivate stickies
Soft reset	<code>aou_con0[4]</code> restores CSR defaults and DISABLED state

`b_transport` never calls `wait()`; it adds `access_delay_ns` (APB) or `bridge_delay_ns` (AXI hop) to the annotated delay.

3. TLM contract

Access	Response
APB 32-bit aligned in <code>[0, 0x80)</code>	<code>TLM_OK_RESPONSE</code>
APB wrong size / unaligned	<code>TLM_GENERIC_ERROR_RESPONSE</code>
APB outside window	<code>TLM_ADDRESS_ERROR_RESPONSE</code>
AXI while not ENABLED / no peer	<code>TLM_COMMAND_ERROR_RESPONSE</code>
AXI <code>dest_rp</code> ≥ peer <code>rp_count</code>	<code>TLM_ADDRESS_ERROR_RESPONSE</code>

4. CCI parameters

Name	Default	Mutability
<code>rp_count</code>	1	immutable (1..4)
<code>access_delay_ns</code>	1.0	mutable
<code>bridge_delay_ns</code>	0.0	mutable

5. SMC / SMU VP wiring

Standalone `smc-vp`:

```
cluster.mmio → fabric → periph_router @ 0xC000_4000 → aou_.apb_socket
aou_axi_s      ← idle initiator
aou_.axi_m[0]   → aou_axi_m → stub_aou_remote
aou_.connect_peer(&aou_peer_)
aou_peer_.axi_m[0] → stub_sysmem
```

`smu-vp` (RTL: AoU on SMU `smu_axi_in/smu_axi_out` = xbar `ext_in/ext_out`):

```
cluster.mmio → fabric → periph_router @ 0xC000_4000 → aou_.apb_socket
SEP 0x4000_4000 → sep_ext_to_smc_axi → fabric → same APB window
smu_axi_xbar.ext_out → aou_axi_s → aou_.axi_s[0]
    (SMC output_axi catch-all, or SEP smn_outbound after outbound filter)
aou_.axi_m[0] → aou_axi_m → smu_axi_xbar.ext_in
aou_.connect_peer(&aou_peer_)          // UCIE PHY stub
aou_peer_.axi_m[0] → stub_sysmem        // remote-die SMN
```

`fdi_active_*` signals tied true at elaboration.

```
aou_.irq_o → aou_irq → intagg.src[21] → plic_src_sig[26] → plic_.src_in[26]
    (PLIC source ID 27; aou_peer_.irq_o is unrouted – the peer
    models the remote die and is not locally firmware-visible)
```

Files: `vp/platform/smc/smc_platform.{hpp,cpp}`, `vp/platform/smc/main.cpp`,
`vp/platform/smu/src/smu_platform.cpp`. Firmware base: `SMC_AOU_BASE` in `sw/smc-vp-`
`tests/common/smc_common.h`.